

Parallel and Distributed Systems

Chapter 11: Hardware for Parallel and Distributed Systems

Prof. Dr. Martin Sträßer

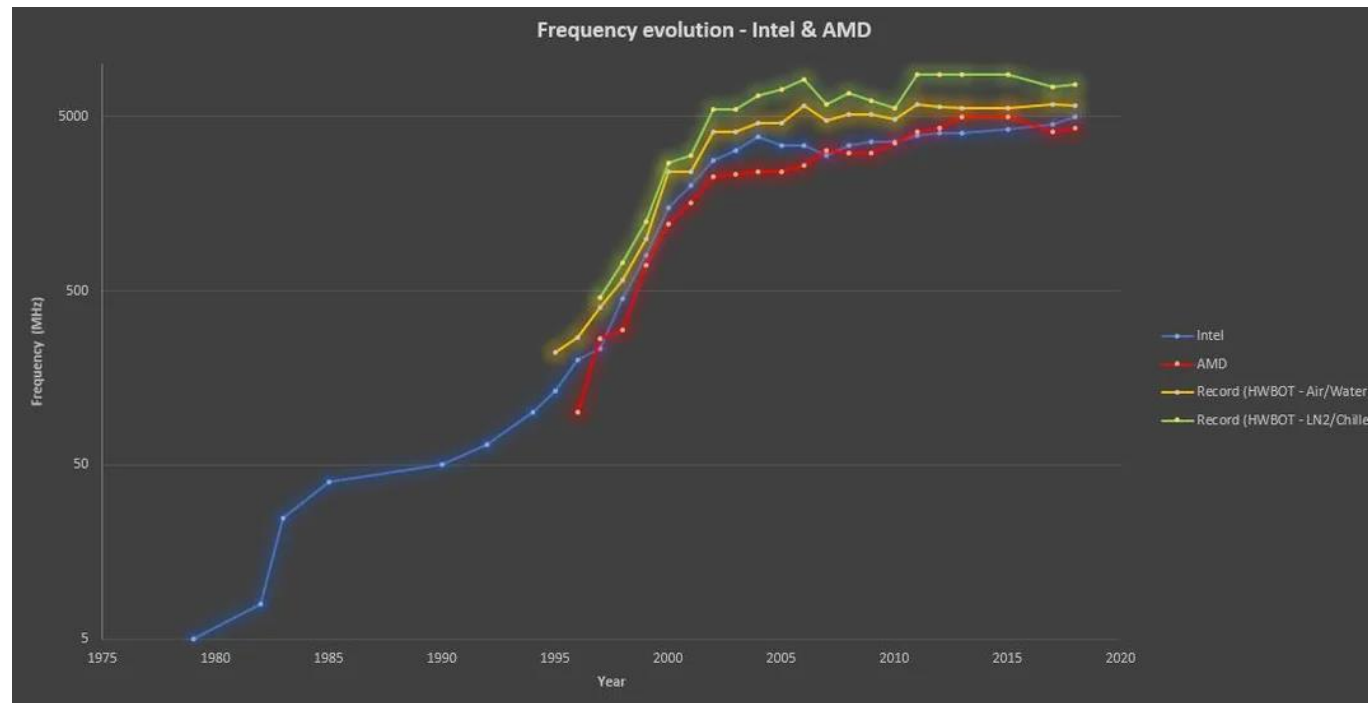


Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap

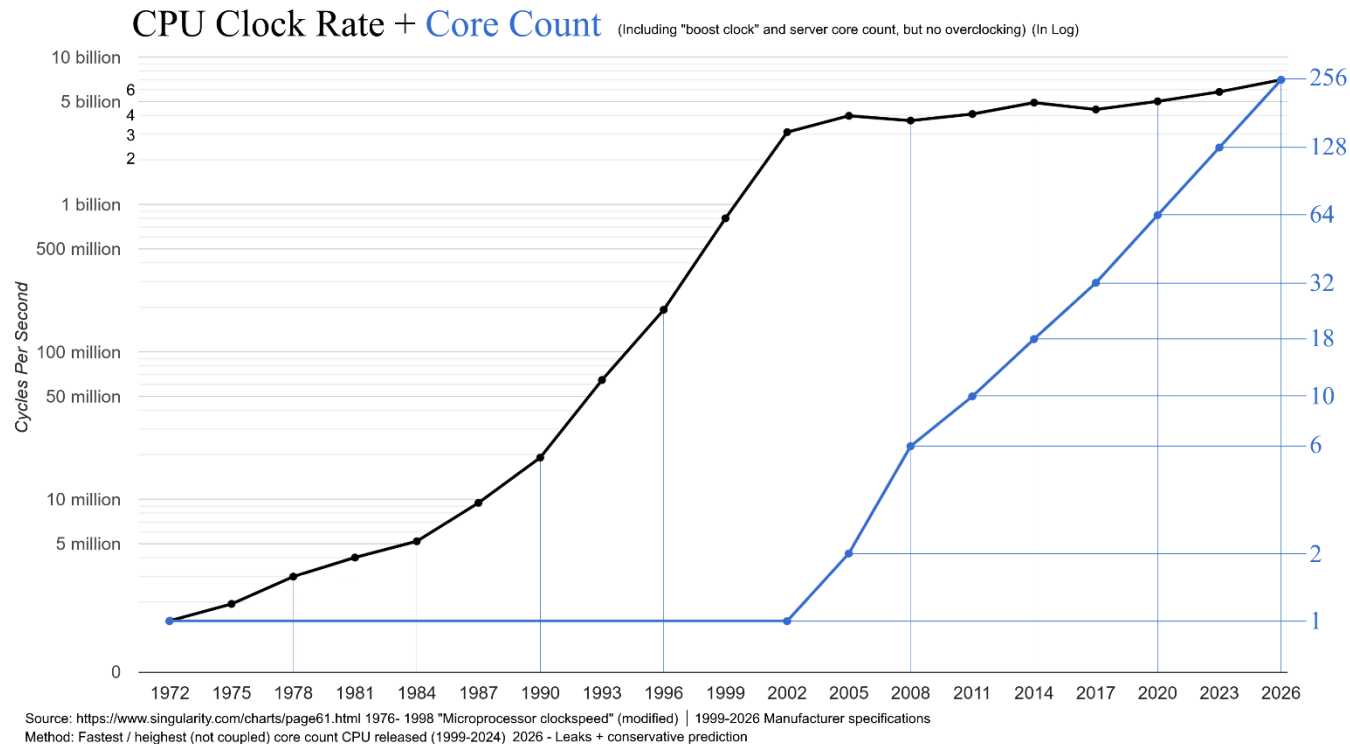
- Which factors prevent us from increasing the frequency (instructions per second) of processors?
- What could we do else to handle more programs at the same time/complete programs faster?





Recap: Logical Processors

- Main problems: Heat and energy consumption
- Solution: Multiple “processors” within a single CPU
- We call these multiple processors within a single CPU *logical processors* or *cores*





Hardware for Parallel and Distributed Systems

- We can not just place multiple processors next to each other and say “work in parallel!”
- Four different ways of hardware support for parallelism
 - Tightly-coupled multiprocessors/multicore processors
 - Manycore processors and tile architectures
 - General-purpose computing on graphics processing unit (GPGPU)
 - Loosely-coupled multiprocessors



Overview

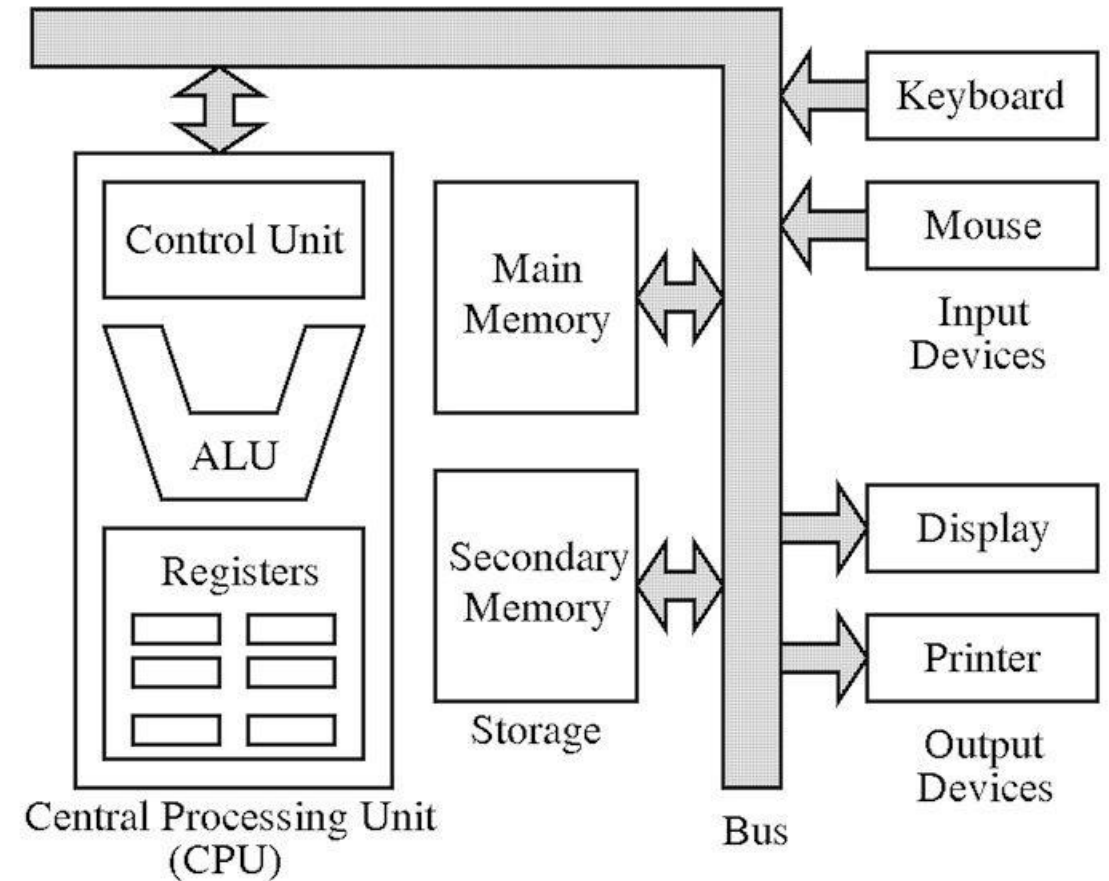
- Tightly-Coupled Multiprocessors and Multicore Processors
- Synchronization Mechanisms and the CPU
- Manycore Processors and Tile Architecture
- General-Purpose Computing on Graphics Processing Unit (GPGPU)
- Loosely-Coupled Multiprocessors

Tightly-Coupled Multiprocessors and Multicore Processors



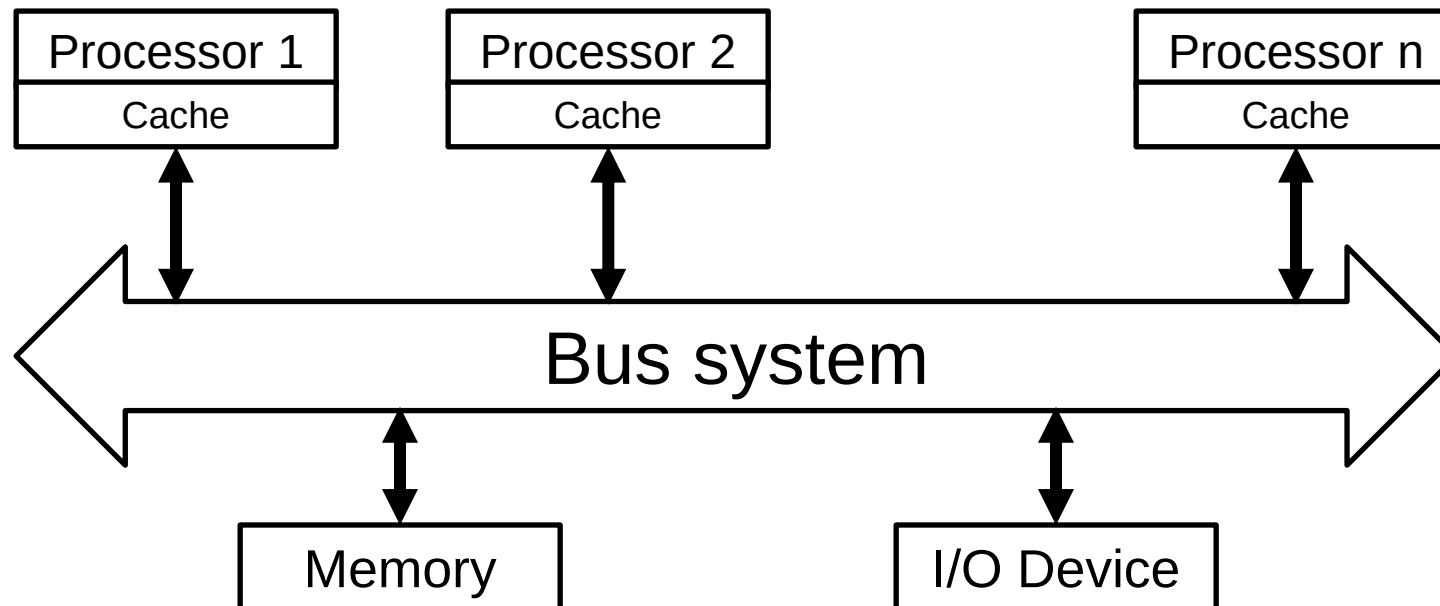
Von Neumann Architecture

- Nearly all modern computers are based on Von Neumann Architecture
- Components
 - Arithmetic Logical Unit (ALU)
 - Control Unit
 - Bus System (control, address, data bus)
 - Main and secondary memory
 - I/O devices



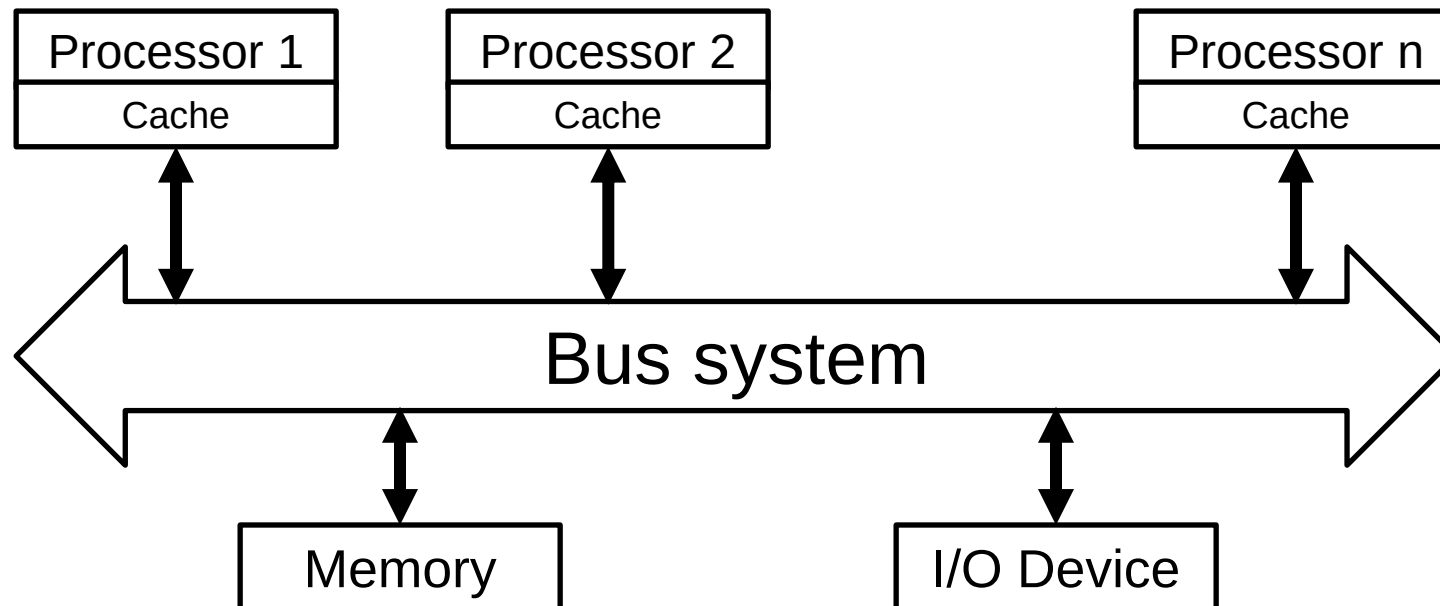
Tightly-Coupled Multiprocessor

- The easiest way of integrating multiple CPUs is to connect additional CPUs to the bus system
- Problem: Von Neumann Bottleneck



Tightly-Coupled Multiprocessor

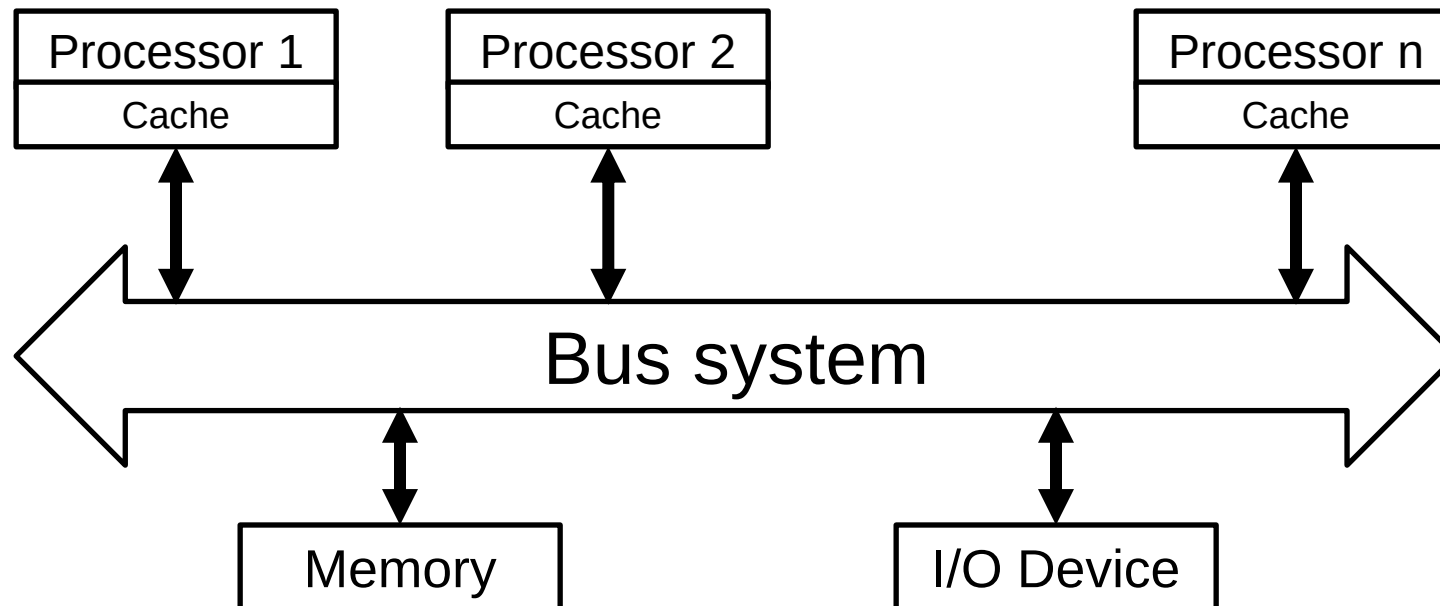
- Advantage: Due to shared memory we can use our known synchronization mechanisms (locks, semaphores, ...) and communication mechanisms





Von Neumann Bottleneck

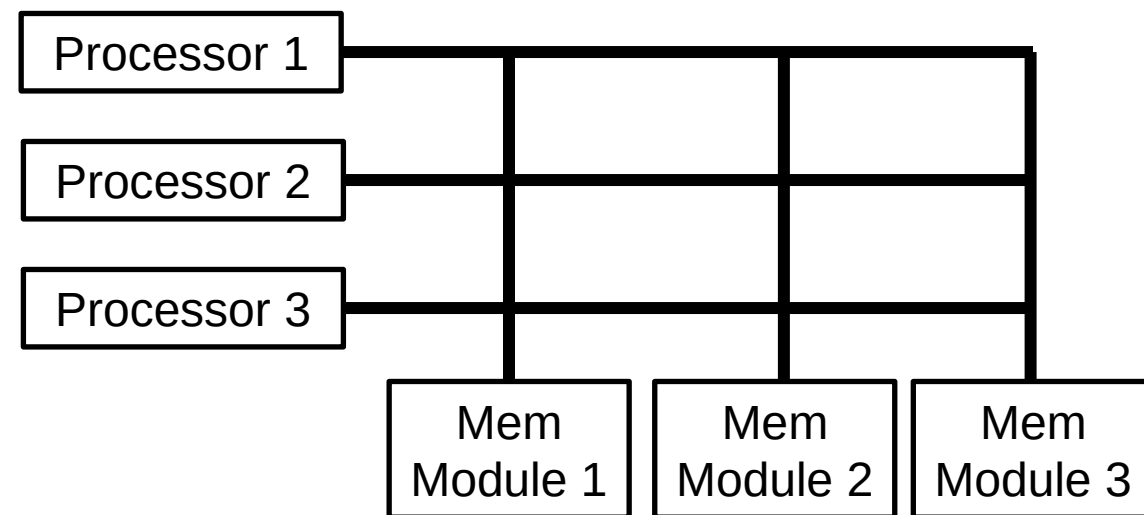
- Von Neumann Bottleneck describes that the CPU frequency is higher than the memory access rate (bus frequency)
- Therefore the CPU is slowed down
- Mitigation: Caches
- But even with caches amount of CPUs limited between 8 and 64





Improving the Bus System

- Modern machines do not use a single line for CPU-memory communication
- Improvement option 1: Crossbar Switches
 - Memory is divided into k modules
 - Crossbar switch can connect n CPUs with k memory modules using $n*k$ switches
 - Non-blocking if CPUs access different memory modules
 - Problem: Does not scale well



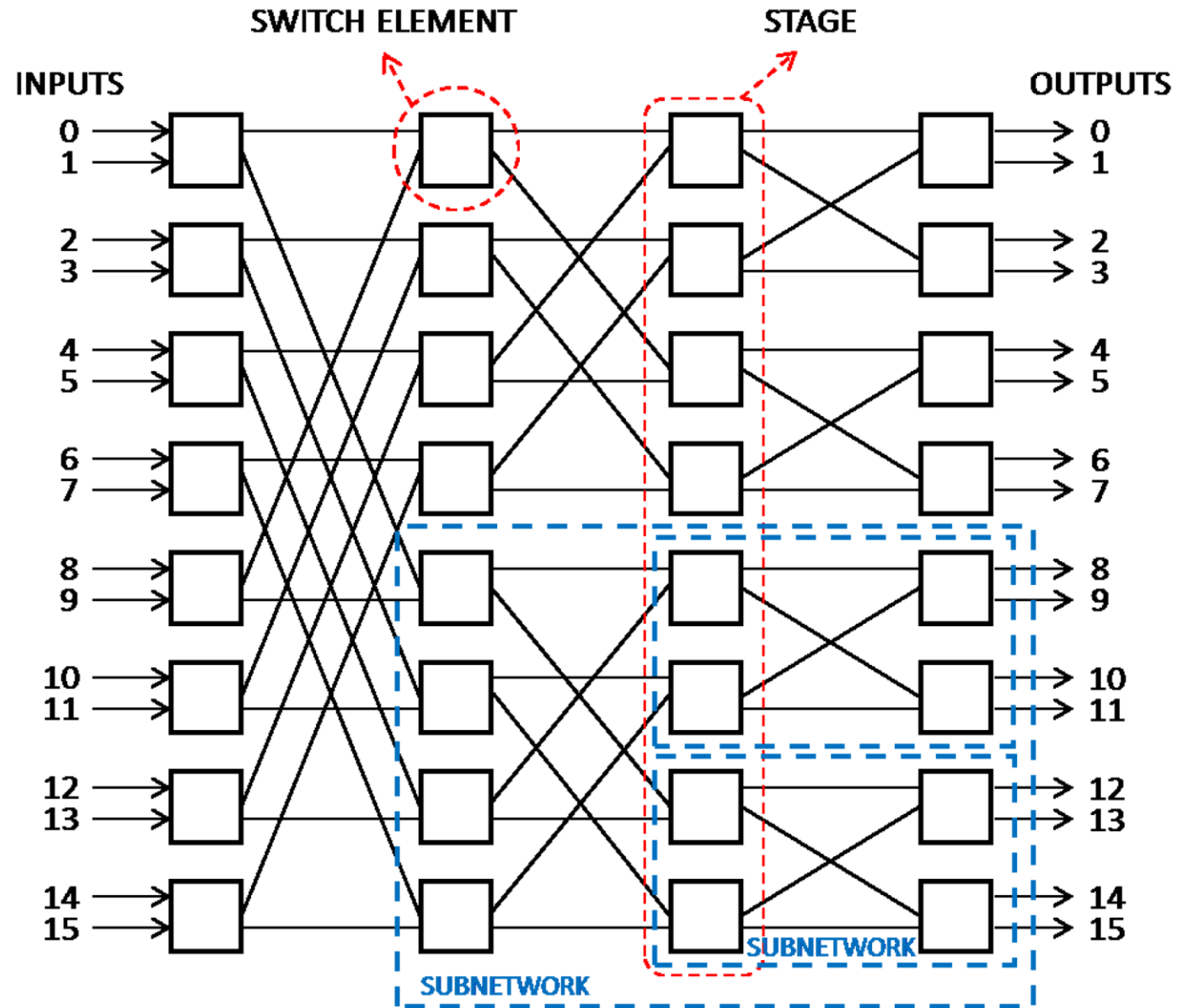


Improving the Bus System

- Improvement option 2: multistage networks
 - Connects n CPUs to n memory modules
 - Multiple layers between CPU and memory
 - If n is a power of two, we need $\log n$ layers with $n/2$ switches each
 - Switches are controlled using a control bit (0 – connect to upper path, 1 – connect to lower path)
 - Can be blocking if bad access pattern
 - Variants of these networks in practice include AMD Infinity Fabric and Intel UPI Mesh Interconnect



Multistage Networks





Multithreading

- Now that we have discussed the CPU-memory connection, let us discuss how a CPU can process multiple threads in parallel
- *Simultaneous Multithreading* is the ability of a processor to process multiple threads in parallel
- Often also referred to as *Hyperthreading* (Intel)
- Two foundations
 - Instruction-Level Parallelism
 - Thread-Level Parallelism



Instruction-Level Parallelism

- Instructions from a common small, sequentially executed program part (e.g., a function) can be executed in parallel if they are independent of each other (e.g., do not use common variables)
- A processor that is able to do so is a *superscalar processor*
- We can distinguish
 - In-order execution: Instructions are processed in the order the compiler specified them. They also complete in this order. Longer running instructions may cause waiting of other instructions.
 - Out-of-order execution: Instructions are dynamically scheduled to increase throughput (standard for CPUs, except microcontrollers)

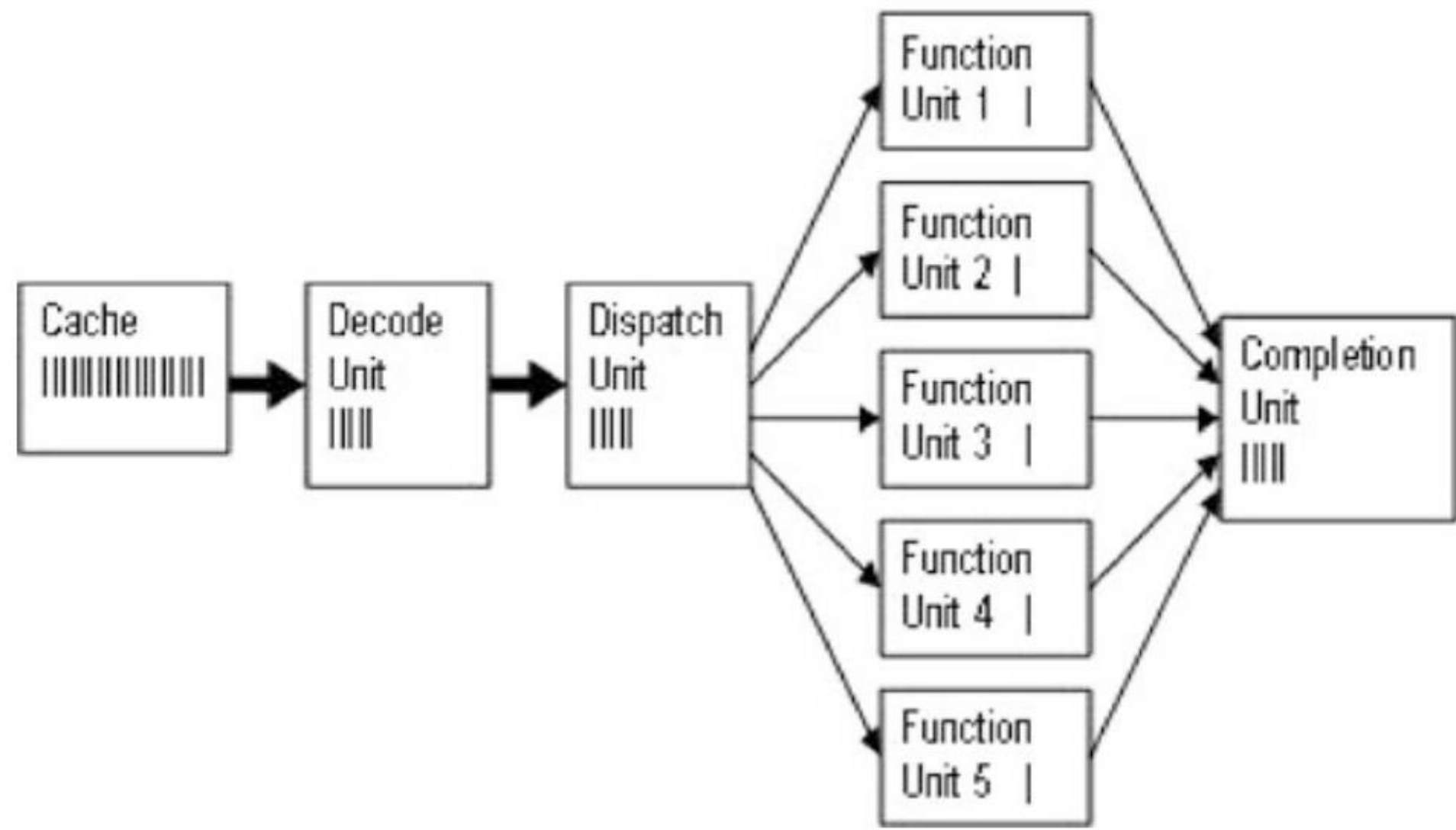


Instruction-Level Parallelism

- Modern CPUs work based on the pipeline pattern
- Subtasks are
 - Fetch: Load instructions from memory
 - Decode: Determines which kind of instructions should be executed (ALU, memory, ...), which registers are needed, how and where should results be stored, may apply branch prediction
 - Dispatch: Analyzes instructions for dependencies and may reorder instructions to increase pipeline throughput
 - Execution: Multiple concurrent execution units execute instructions
 - Completion: Writes results and checks for interrupts/errors



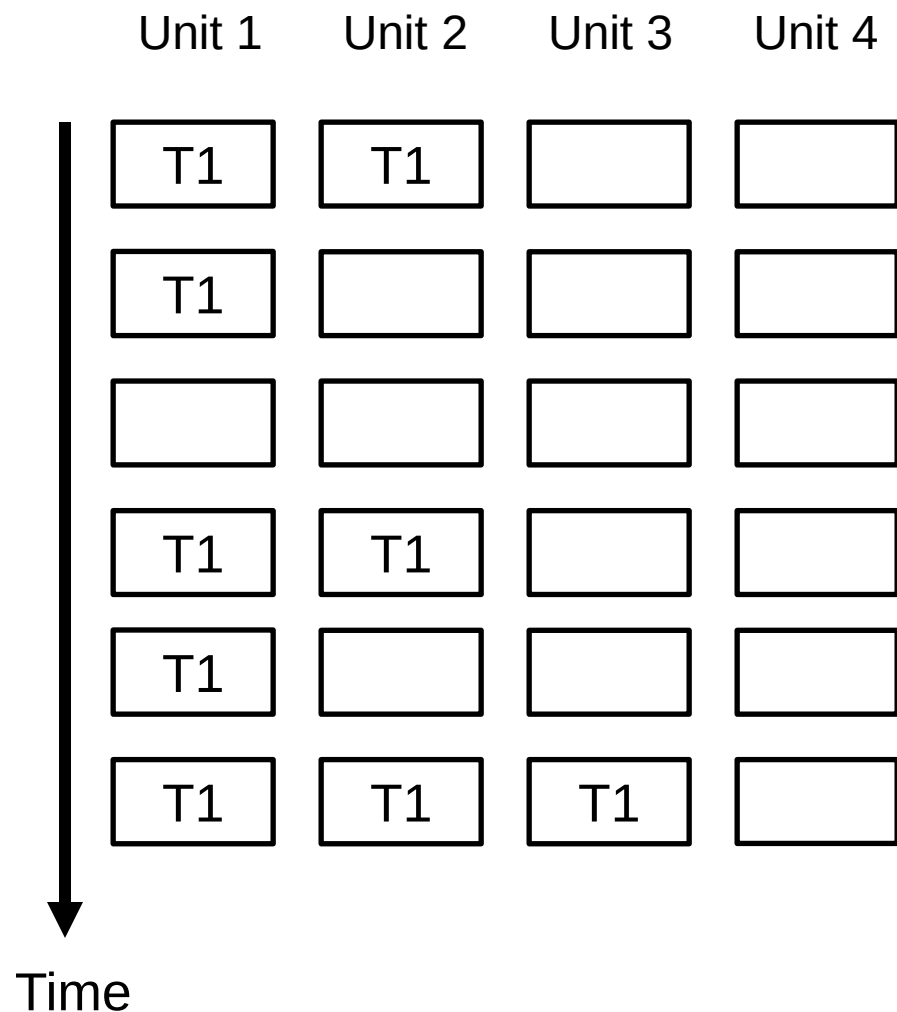
Instruction-Level Parallelism





Instruction-Level Parallelism

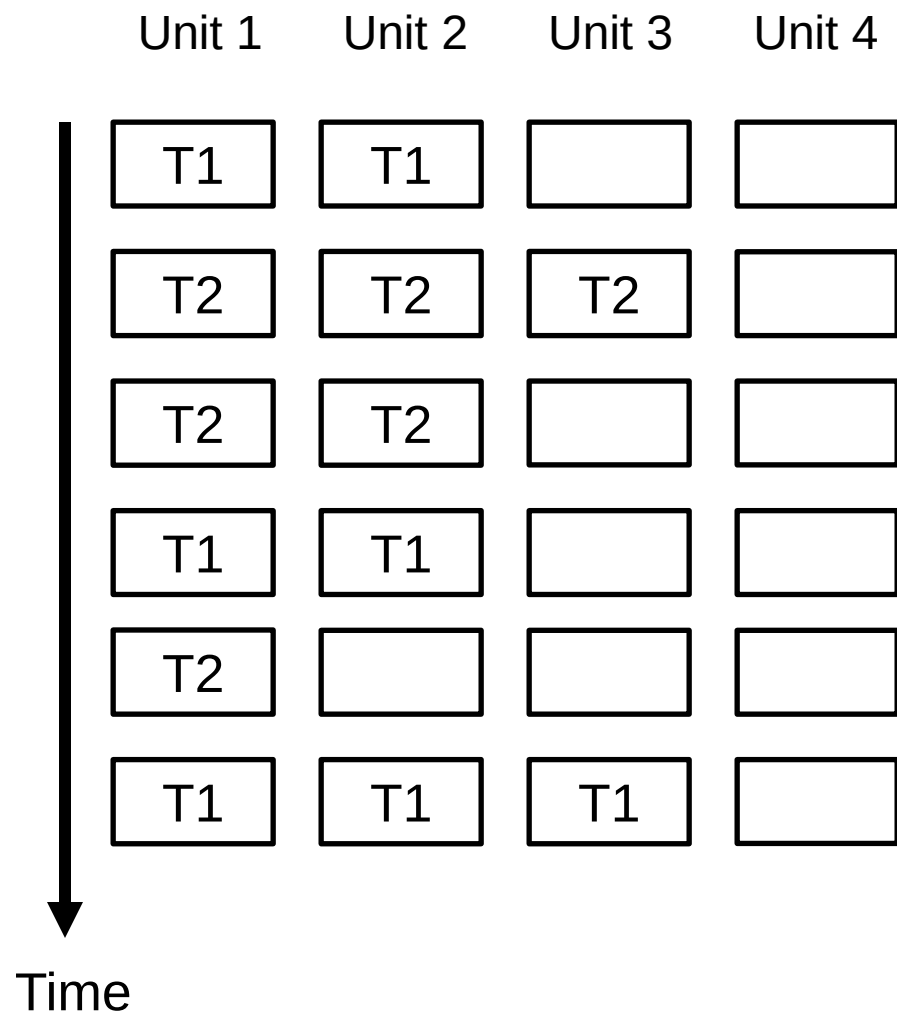
- Throughput depends on
 - Number of identified instructions that can run in parallel
 - Completion time of instructions (some instructions execute longer than one CPU cycle, e.g., memory reads)
- In instruction-level parallelism, parallel instructions always originate from a single thread T1
- Example: Realistic throughput with 4 function units





Thread-Level Parallelism

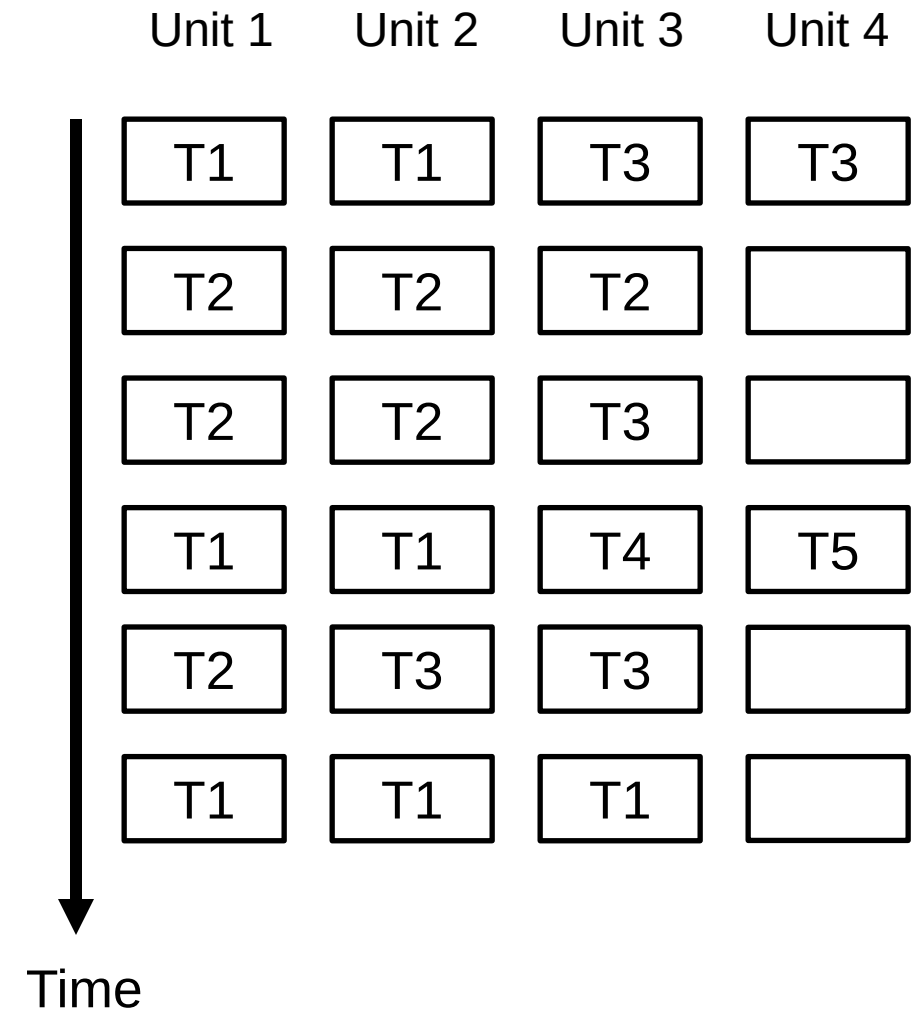
- With thread-level parallelism, the CPU may switch the context between cycles
- Other threads can be executed concurrently





Simultaneous Multithreading

- In simultaneous multithreading, different instructions of different threads can be executed in parallel
- In combines instruction-level and thread-level parallelism





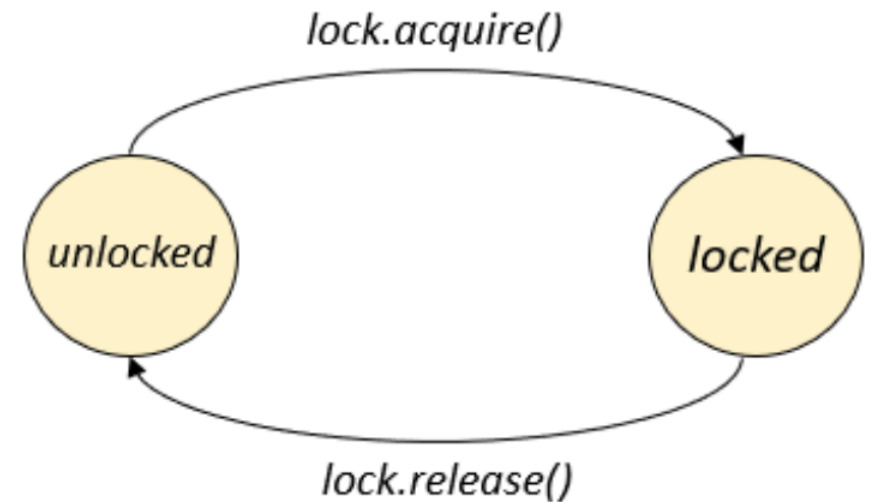
Multicore Processors

- The biggest performance boosts are reached by replication
- Multiprocessor (multiple chips) vs. cores (multiple CPUs, one chip)
- Multiple cores may
 - Share a common Fetch- and/or Decode-Unit (Vector machines)
 - Have independent Fetch- and Decode-Units (Multicore processors)
- Multicore processors may
 - Have n equal cores (symmetric multiprocessing), common for general-purpose laptops, servers etc.
 - Special-purpose (unequal) cores, common for embedded systems like automotive or medical devices
- Multicore processors often have a 3-layered caching system (private core cache, shared caches)

Synchronization Mechanisms and the CPU

Recap: Locks

- Simplest synchronization mechanism
- When entering a critical section, lock has to be acquired
- When leaving a critical section, lock is released
- Lock can be acquired by one thread at a time
- Acquire and release operations are atomic
- Native support by operating system





Hardware Realization of Locks

- Locks ensure mutual exclusion, meaning that only one thread/process can enter a critical section at a time
- We learned that in multiprocessor systems, context switches may occur between CPU cycles
- In order to acquire a lock, we need to **read** the current state of the lock and potentially **change** the state of the lock
- One way of realizing locks in hardware is to execute special (comparatively complex) instructions that are indivisible

Test and Set (TAS)

- Test and Set (TAS) allows indivisible read and write operation
- Variables:
 - Can_Not_Enter: boolean describing whether the thread can not enter the critical section (= return value of operation)
 - Active: boolean describing whether critical section is locked
- Logic (C++-like representation):

```
bool TAS (bool& Active) {  
    bool Can_Not_Enter = Active;  
    if (Active == false)  
        Active = true;  
    return Can_Not_Enter;  
}
```

Exchange (XCHG)

- Exchange (XCHG) allows indivisible swap the values of two boolean
- Variables:
 - Can_Not_Enter: boolean describing whether the thread can not enter the critical section (= return value of operation)
 - Active: boolean describing whether critical section is locked
- Logic:

```
void XCHG (bool& Active, bool& Can_Not_Enter)
{
    bool temp = Active;
    Active = Can_Not_Enter;
    Can_Not_Enter = temp;
}
```

Spinlocks

- Using TAS or XCHG we can realize spinlocks (active waiting)
- Spinlocks make sense at CPU level as resources are often locked for a very short time and a context switch might be more costly (cache contents are changed as well)

```
void spin_lock (bool& Active) {  
    bool Can_Not_Enter = true;  
    while (Can_Not_Enter) {  
        Can_Not_Enter = TAS(Active);  
        // or: XCHG(Active, Can_Not_Enter);  
    }  
}
```

```
void spin_unlock (bool& Active) {  
    Active = false;  
}
```



Semaphores

- Given the `spin_lock` and `spin_unlock` operations, we can now implement a semaphore
- A semaphore is a data structure containing:
 - counter: Integer representing how many processes can enter critical section now
 - Queue: Queue of waiting processes



Semaphores

```
void acquire(bool& Active, Semaphore& s) {  
    spin_lock(Active);  
    s.counter = s.counter - 1;  
    if (s.counter < 0) {  
        s.Queue.enqueue(get_running_pcb());  
        spin_unlock(Active);  
        sleep();  
    } else {  
        spin_unlock(Active);  
    }  
}
```

```
void release (bool& Active, Semaphore& s) {  
    spin_lock(Active);  
    s.counter = s.counter + 1;  
    if (s.counter <= 0) {  
        wake(s.Queue.dequeue());  
    }  
    spin_unlock(Active);  
}
```

Manycore processors and tile architectures



From Multicore to Manycore Processors

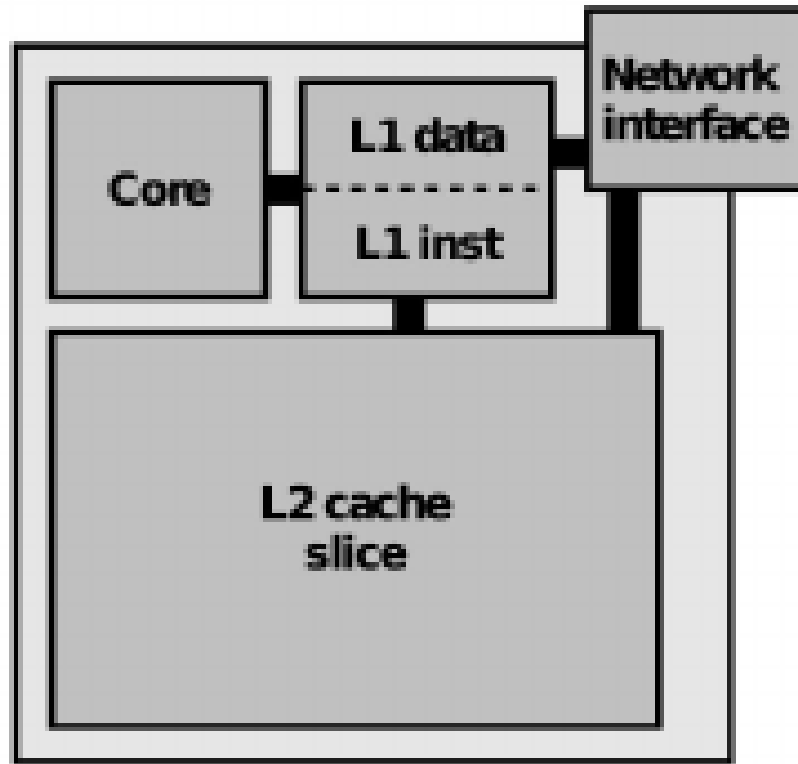
- Multicore processors
 - Use few (e.g., 32) heavyweight cores
 - Often have large caches and are optimized for latency (low execution times)
 - Good for general purpose workloads (e.g., laptops) and irregular code (many branches, not predictable)
- Manycore processors
 - Use many (e.g., 1000s) simple, small, energy-efficient cores
 - Optimized for throughput (many tasks in parallel)
 - Use specialized programming models
 - Critical factor: Coordination of operations and having the right data at the right place



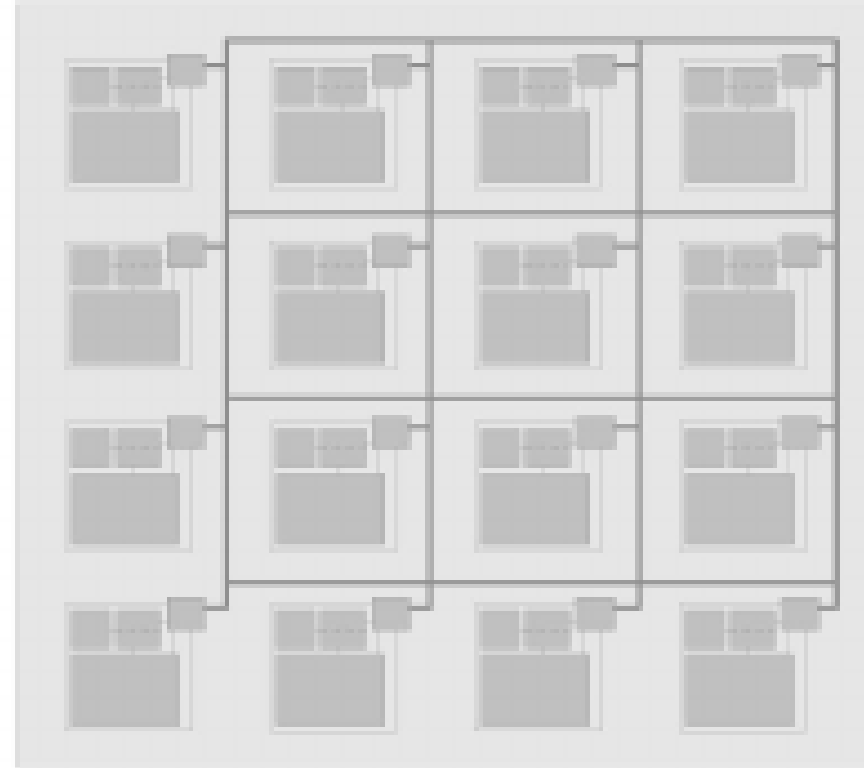
From Multicore to Manycore Processors

- Multicore processors cannot contain thousands of cores, reasons:
 - Shared bus system
 - Often shared caches
- Manycore processors replicate everything locally
- A tile is a small, self-contained compute unit that may include one or few cores, local cache (typically two layers), local memory controllers, and a router or network interface
- Tiles are connected using an *on-chip network*
- Tiles communicate using packet-based messaging

Tile Architecture



(a)



(b)



Design Trade-Offs for Tile Architectures

- Tile size (e.g., for power delivery)
- Topology of on-chip networks
- Memory hierarchy and access
- Programming complexity

General-purpose computing on graphics
processing unit (GPGPU)



Graphical Processing Units (GPUs)

- As the name suggests, originally used for graphics processing and gaming only
- Nowadays famous for AI computations
- Why?
 - GPUs are excellent for Single Instruction Multiple Data (SIMD) workloads
 - In other words: They are able to perform the same operation on huge chunks of data in parallel
 - AI computations are essentially a lot of vector/matrix operations



CPU vs. GPU

- CPU
 - CPU cores allows for executing complex logic (e.g., branching)
 - Often comparatively few cores (e.g., 16)
- GPU
 - Based on tile architecture
 - Thousands of simpler cores that can only execute few set of instructions
 - Complex instructions (e.g, branching) are super-expensive
 - Best when all threads execute same operation
- Libraries like CUDA offload computation tasks from the CPU (runs the *host program*) to the GPU (called *device*) if desirable
- GPU also has memory, best performance when all data fit into GPU memory only



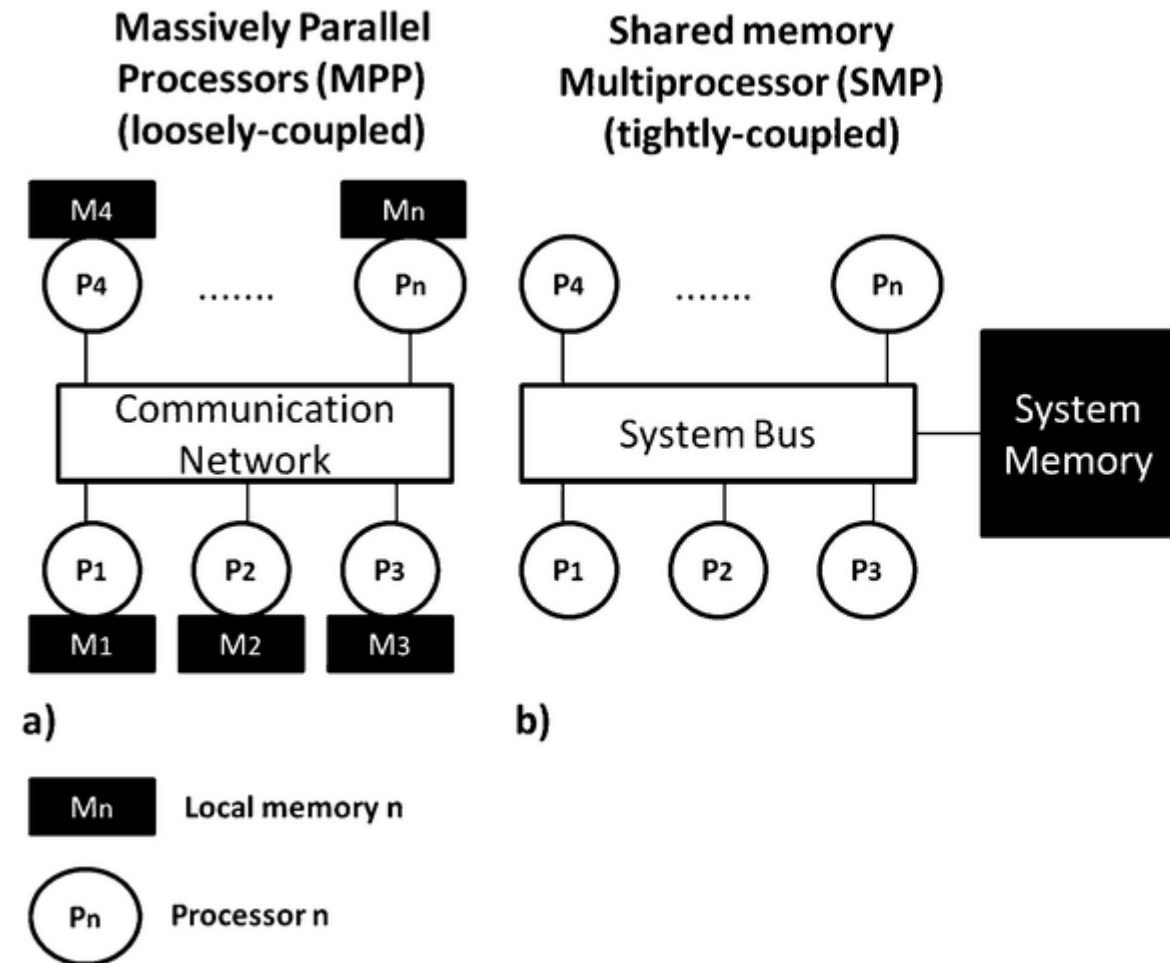
Using GPUs in Programs

- Requires special libraries
- Mainly developed for C++ and Python
- Most famous one is Common Unified Device Architecture (CUDA) by Nvidia
- Others include
 - OpenCL
 - OpenMP Offloading
 - HIP by AMD

Loosely-coupled multiprocessors

Tightly-Coupled vs. Loosely-Coupled

- Tightly-coupled multiprocessors
 - All processors access common shared memory
 - This principle is called Unified Memory Access (UMA)
 - Limited but predictable performance
- Loosely-coupled multiprocessors
 - Every processor has own local memory
 - Processor-memory pairs connected via inter-connection network
 - Better scalability





Non-Uniform Memory Access (NUMA)

- Processors may need to access their local memory sometimes, but also “remote” memory (local memory of other processors)
- Accessing local memory is obviously faster than accessing “remote” memory
- Non-Uniform Memory Access (NUMA)
- All local memories of all processors are summarized in a single virtual address space (Distributed Shared Memory)